



COURSE CODE/TITLE: CPE 010

SECTION: CPE21S1

DATE PERFORMED: 07-29-26

DATE SUBMITTED: 07-29-26

NAME: Paula Esguerra

INSTRUCTOR: Engr. Jimlord Quejado

HOA NO. 3 Linked Lists

1. PROCEDURE OUTPUT

Screenshot

```
C:\Users\Paramount Laptop 5 X + v
Testing of Traversal:
P->A->U->E

Testing of sllinsertHead
Q->P->A->U->E

Testing of sllGeneralInsert
Q->P->A->X->U->E

Testing of sllInsertEnd
Q->P->A->X->U->E->W

Testing deleting node
P->A->X->U->E->W

P->A->U->E->W

P->A->U->E

Deleting list
The list is empty
```

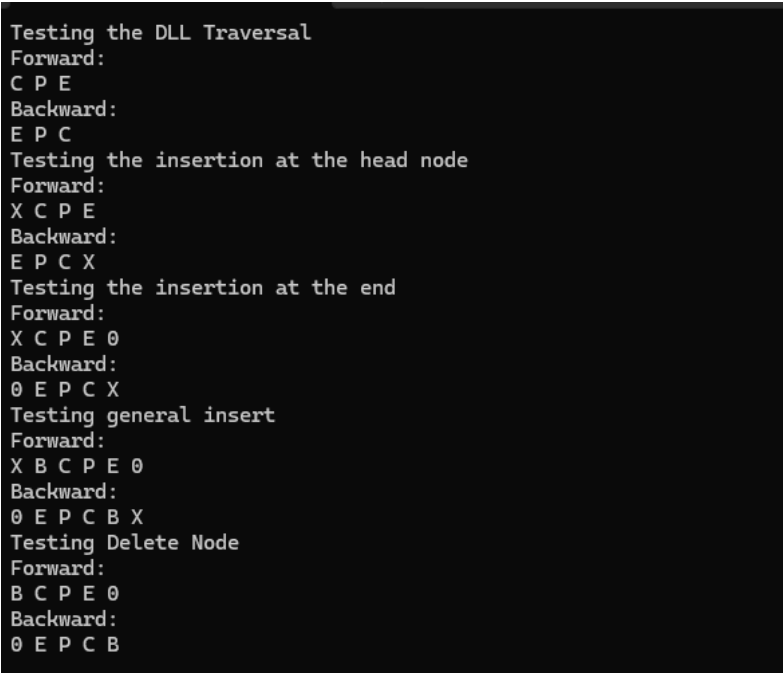
	 <pre> Testing the DLL Traversal Forward: C P E Backward: E P C Testing the insertion at the head node Forward: X C P E Backward: E P C X Testing the insertion at the end Forward: X C P E 0 Backward: 0 E P C X Testing general insert Forward: X B C P E 0 Backward: 0 E P C B X Testing Delete Node Forward: B C P E 0 Backward: 0 E P C B </pre>
Discussion	Singly is copied from the activity before because it has the same function that i need, where inserting a node, generally adding, deleting the entire problem or just to delete a node.

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot
-----------	------------

traversal

```

12
13 template <typename T>
14 void SingleListTraversal(SingleList<T>* head) {
15
16     if (head == nullptr) {
17         std::cout << "The list is empty";
18         return;
19     }
20
21     while(head != nullptr) {
22         // PRINT data OF n
23         std::cout<<head->data;
24
25         if(head->next != nullptr) {
26             std::cout << "->";
27         }
28
29         // GO TO NEXT NODE n -> next
30         head = head->next;
31     }
32

```

```

29
30 template <typename T>
31 void dllTraverse(DoubleList<T>* currentNode) {
32
33     //temp variable to store the lat node visited
34     DoubleList<T> *tail;
35
36     //check if the list is empty
37     if(currentNode == nullptr) {
38         std::cout<<"The list is empty" <<std::endl;
39         return;
40     }
41
42     //move forward
43     std::cout << "Forward: \n";
44     while(currentNode != nullptr) {
45         std::cout << currentNode->data << " ";
46         tail = currentNode;
47         currentNode = currentNode->next;
48     }
49

```

```

49
50 //add a next line
51 std::cout << std::endl;
52
53 //move backward
54 std::cout << "Backward: \n";
55
56 while(tail != nullptr){
57     std::cout<< tail->data<<" ";
58     tail = tail->prev;
59 }
60 std::cout<<std::endl;
61
62 }
63

```

Insertion at head

```

35
36 template <typename T>
37 void sllinsertHead(T newData, SingleList<T>** head){
38
39     // 1. Allocate new memory for the new code
40     SingleList<T>* newNode = new SingleList<T>;
41     // 2. Put our data into the new node
42     newNode->data = newData;
43     // 3. Set next to the new node to point to the previous head
44     newNode->next = *head;
45     // 4. Reset head to the new node
46     *head = newNode;
47 }
48
49
50 //inserting at the head of the list
51 template <typename T>
52 void dllInsertHead(T newData, DoubleList<T> ** currentHead){
53     // creating a new node
54     DoubleList<T>* newNode = CreateNewNode(newData);
55
56     //new node should point to the current head
57     newNode->next = *currentHead;
58
59     //current Head should point back to the newNode
60     (*currentHead)->prev = newNode;
61
62     //update the pointer head
63     *currentHead = newNode;
64 }
65

```

Insertion at any part of
the list

```

48
49 template <typename T>
50 void sllGeneralInsert(T newData, SingleList<T>* prevNode) {
51
52     //1. Check if it is the head node (previous node is null)
53     if(prevNode == nullptr){
54         //2. If null, print "Previous node cannot be null."
55         std::cout<<"Previous value cannot be null"<<std::endl;
56         return;
57     }
58     //3. Allocate a new node
59     SingleList<T>* newNode = new SingleList<T>;
60     //4. Store data in the new node
61     newNode->data = newData;
62     //5. Point new node to the node previous node was pointing to
63     newNode->next = prevNode->next;
64     //6. Point previous node to the new node
65     prevNode->next = newNode;
66 }

```

```

100
101 //general insert
102 template <typename T>
103 void dllGeneralInsert(T newData, DoubleList<T>* prevNode) {
104     if(prevNode == nullptr) {
105         std::cout<<"Previous value cannot be null" <<std::endl;
106         return;
107     }
108     DoubleList<T>* newNode = new DoubleList<T>;
109     newNode->data = newData;
110     newNode->next = prevNode->next;
111     newNode->prev = prevNode;
112     newNode->next->prev = newNode;
113     prevNode->next = newNode;
114 }
115
116
117

```

Insertion at the end

```

67
68 template <typename T>
69 void sllInsertEnd(T newData, SingleList<T>** head) {
70     // 1. Allocate a new node
71     SingleList<T>* newNode = new SingleList<T>;
72
73     // 3. Store data in new node
74     newNode->data = newData;
75     // 4. Point to null
76     newNode->next = nullptr;
77
78     // 2. Dereference the head node
79     SingleList<T>* currentNode = *head;
80
81     // 5. Traverse the list
82     while(currentNode->next != nullptr) {
83         currentNode = currentNode->next;
84     }
85
86     // 6. Point the next of the current node to the new node
87     currentNode->next = newNode;
88 }
89
90

```

	<pre> 81 82 template <typename T> 83 84 void dllInsertEnd(T newData, DoubleList<T> * currentHead){ 85 //create a new node 86 DoubleList<T>* newNode = CreateNewNode(newData); 87 88 //traverse until we reach the last node 89 while(currentHead->next != nullptr){ 90 currentHead = currentHead->next; 91 } 92 93 //connect the last node to the new node 94 currentHead->next = newNode; 95 96 //connect the new node back to last node 97 newNode->prev = currentHead; 98 99 } 100 </pre>
Deletion of a node	<pre> 89 90 template <typename T> 91 void sllDeleteNode(T findData, SingleList<T>** head){ 92 93 // if there is nothing to delete 94 if (*head == nullptr) return; 95 96 SingleList<T>* currNode = *head; 97 SingleList<T>* prevNode = nullptr; 98 99 // searching the node: 1. the node does not exist + we found data 100 while (currNode != nullptr && currNode->data != findData){ 101 prevNode = currNode; 102 currNode = currNode->next; 103 } 104 105 // if data not found currNode == nullptr in the while loop 106 if(currNode == nullptr){ 107 std::cout << "the data was not found\n" << std::endl; 108 return; 109 } 110 111 // if data was found 112 if(prevNode == nullptr){ 113 *head = currNode->next; 114 } else { 115 prevNode->next = currNode->next; 116 } 117 118 delete currNode; 119 } 120 121 </pre>
Deletion of a List	

```

120 }
121
122 template <typename T>
123 void sllDeleteList(SingleList<T>** head) {
124     SingleList<T>* current = *head;
125
126     while(current != nullptr) {
127         SingleList<T>* temp = current;
128         current = current->next;
129         delete temp;
130     }
131
132     *head = nullptr;
133 }
134
135 #endif
136
137

```

```

118 //delete a node
119 template <typename T>
120 void dllDeleteList(T findData, DoubleList<T> ** head) {
121     if (*head == nullptr) return;
122     DoubleList<T>* currentNode = *head;
123     DoubleList<T>* prev = nullptr;
124
125     while (currentNode != nullptr && currentNode->data != findData) {
126         prev = currentNode;
127         currentNode = currentNode->next;
128     }
129
130     if(currentNode == nullptr) {
131         return;
132     }
133
134     if(currentNode == *head) {
135         *head = currentNode->next;
136         if (*head != nullptr) {
137             (*head)->prev = nullptr;
138         }
139     }
140
141     else {
142         prev->next = currentNode->next;
143         if (currentNode->next != nullptr) {
144             currentNode->next->prev = prev;
145         }
146     }
147     delete currentNode;
148 }
149
150 #endif
151

```

Table 3-3. Code and Analysis for Singly and Double Linked Lists (respectively)

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```
class.cpp main.cpp
1  #ifndef PLAYLIST_H
2  #define PLAYLIST_H
3
4  #include <string>
5
6  // 1. Structure Design
7  struct SongNode {
8      std::string title;
9      SongNode* next;
10     SongNode* prev;
11
12     SongNode(std::string songTitle);
13 };
14
15 // 2. Class Interface Declaration
16 class Playlist {
17 private:
18     SongNode* head;
19     SongNode* current;
20
21 public:
22     Playlist();
23     ~Playlist();
24 }
```


Debug

class.h

main.cpp

```
10     SongNode* prev;
11
12     SongNode(std::string songTitle);
13 };
14
15 // 2. Class Interface Declaration
16 class Playlist {
17 private:
18     SongNode* head;
19     SongNode* current;
20
21 public:
22     Playlist();
23     ~Playlist();
24
25     void insertSong(std::string title);
26     void removeSong(std::string title);
27     void nextSong();
28     void previousSong();
29     void playAllSongs();
30 };
31
32 #endif // PLAYLIST_H
33
```

Compiler | Output | Run | Stop

class.h main.cpp

```
1  #include <iostream>
2  #include "Playlist.h" // Links the header file definitions
3
4  // SongNode Constructor Implementation
5  SongNode::SongNode(std::string songTitle) {
6      title = songTitle;
7      next = nullptr;
8      prev = nullptr;
9  }
10
11 // Playlist Constructor Implementation
12 Playlist::Playlist() {
13     head = nullptr;
14     current = nullptr;
15 }
16
17 // Add song to the end of the playlist
18 void Playlist::insertSong(std::string title) {
19     SongNode* newNode = new SongNode(title);
20
21     if (head == nullptr) {
22         head = newNode;
23         head->next = head;
24         head->prev = head;
```

es  Compile Log  Debug  Find Results  Close

```

25     current = head;
26     std::cout << "Added first song: " << title << "\n";
27     return;
28 }
29
30 SongNode* tail = head->prev;
31 tail->next = newNode;
32 newNode->prev = tail;
33 newNode->next = head;
34 head->prev = newNode;
35
36     std::cout << "Added: " << title << "\n";
37 }
38
39 // Remove a song matching the title
40 void Playlist::removeSong(std::string title) {
41     if (head == nullptr) {
42         std::cout << "Playlist is empty.\n";
43         return;
44     }
45
46     SongNode* currNode = head;
47     bool found = false;
48
49     do {
50         if (currNode->title == title) {
51             found = true;
52             break;
53         }
54         currNode = currNode->next;
55     } while (currNode != head);

```

```

54     currNode = currNode->next;
55 } while (currNode != head);
56
57 if (!found) {
58     std::cout << "Song " << title << " not found.\n";
59     return;
60 }
61
62 if (currNode->next == currNode) {
63     head = nullptr;
64     current = nullptr;
65 } else {
66     if (currNode == head) head = head->next;
67     if (currNode == current) current = current->next;
68
69     currNode->prev->next = currNode->next;
70     currNode->next->prev = currNode->prev;
71 }
72
73 delete currNode;
74 std::cout << "Successfully removed: " << title << "\n";
75 }
76
77 void Playlist::nextSong() {
78     if (current == nullptr) return;
79     current = current->next;
80     std::cout << "Skipped Forward. Now Playing: " << current->title << "\n";
81 }
82
83 void Playlist::previousSong() {
84     if (current == nullptr) return;
85     current = current->prev;

```

```

class.h  main.cpp
82
83 void Playlist::previousSong() {
84     if (current == nullptr) return;
85     current = current->prev;
86     std::cout << "Skipped Backward. Now Playing: " << current->title << "\n";
87 }
88
89
90 void Playlist::playAllSongs() {
91     if (head == nullptr) {
92         std::cout << "Playlist is empty.\n";
93         return;
94     }
95
96     std::cout << "\n--- Playing all tracks ---\n";
97     SongNode* temp = head;
98     do {
99         std::cout << "Now Playing: " << temp->title << "\n";
100        temp = temp->next;
101    } while (temp != head);
102    std::cout << "-----\n";
103 }
104
105 // Destructor
106 Playlist::~Playlist() {
107     if (head == nullptr) return;
108     head->prev->next = nullptr;
109     SongNode* currNode = head;
110     while (currNode != nullptr) {
111         SongNode* nextNode = currNode->next;
112         delete currNode;
113         currNode = nextNode;
114     }

```

Conclusion:

The activity demonstrates how a circular doubly linked list provides the optimal structural foundation for a loop-enabled music player. Standard arrays fail to handle continuous looping naturally, whereas a circular structure ensures that moving past the last song seamlessly cycles back to the first. Incorporating both next and prev pointers allows intuitive navigation to skip both forward and backward efficiently. Finally, implementing a customized class destructor ensures that the circular structure is broken apart before memory deallocation, successfully preventing system memory leaks during song insertion and removal operations.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Idea Generation, Code assistance, and Explaining of concepts



COMPUTER
ENGINEERING

QUEZON CITY

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.